

### Prompt 1: Assignment and Skeleton Analysis

Prompt: Please read the attached assignment PDF and the provided empty skeleton notebook (ass1\_empty\_workbook.ipynb). Your task is to analyze both documents and identify every required code implementation component across Tasks 1 through 4. **CRITICAL INSTRUCTION:** Do not attempt to answer any of the conceptual or text-based assignment questions. Your assistance must be strictly confined to code implementation, architecture layout, and execution logic. Please provide a clean, highly structured, and concise implementation plan that addresses:

- The expected dataset structures and mapping strategies.
  - Required model architectures and loss configurations.
  - Evaluation scenarios, validation loops, and expected performance metrics.
    - Necessary visualizations and plotting layouts.
    - Anomaly detection frameworks for Task 4.
  - The target layout structure of the final notebook.
- 

### Prompt 2: Reproducible Workflow Setup (notebook\_builder.py)

Prompt: I need to design a completely reproducible workflow for generating our final submission notebook from the empty skeleton notebook. Please write a robust standalone Python script named notebook\_builder.py. The script must satisfy the following technical requirements:

1. Use the nbformat library to systematically read ass1\_empty\_workbook.ipynb.
  2. Maintain structural parity: preserve every single original markdown heading, structural text cell, and empty cell placeholder from the skeleton.
  3. Inject our custom-implemented code solutions *only* into the designated empty code cell blocks corresponding to each task.
  4. Completely clear out all existing outputs, visual artifacts, metadata, and sequential execution counts across all cells to ensure a perfectly clean slate.
  5. Output and save the newly assembled, unexecuted notebook as ass1\_final\_submission.ipynb.
- 

### Prompt 3: Global Setup Cell Implementation

Prompt: Please generate the code for the global configuration and setup cell of the notebook. It must include:

1. Clean, organized imports covering: PyTorch (torch, torch.nn, torch.nn.functional, torch.utils.data), torchvision transforms and utilities, kagglehub, PIL.Image, numpy, matplotlib.pyplot, pathlib.Path, and collections utilities.
  2. Dataset Pinning: Programmatic downloading or pinning of the dataset utilizing the handle moltean/fruits/versions/87.
  3. Global Hyperparameters: Explicitly define the following global constants:

```
SEED = 6
BATCH_SIZE = 32
IMAGE_SIZE = 64
EMBED_DIM = 128
```
  4. Determinism: Write a comprehensive set\_seed(seed) utility function that sets seeds across random, numpy, torch, and configures torch.backends.cudnn.deterministic = True and torch.backends.cudnn.benchmark = False. Ensure this function is called immediately.
  5. Device Management: Implement a dynamic device selection variable that sets the runtime to CUDA if a GPU is available, fallback to CPU otherwise.
- 

### Prompt 4: Task 1: AppleDataset Implementation

Prompt: Write a custom PyTorch Dataset class named AppleDataset for Task 1. The implementation must conform to these strict constraints:

1. Use kagglehub to fetch the pinned dataset (moltean/fruits/versions/87) and target the inner fruits-360\_original-size/fruits-360-original-size directory.

2. The `__init__` constructor must accept a split argument (either 'Training' or 'Validation') and a class\_subset string filter (which can be 'main', 'new', or 'all').
  3. Directory Filtering: Programmatically scan the directory and extract only folder names that explicitly start with the prefix apple.
  4. Class Splitting: Deterministically sort and shuffle these apple item folders using our fixed seed (SEED = 6). Divide them such that the first 20 varieties are classified as main apple items, and the remaining 10 varieties are classified as new apple items.
  5. The class\_subset parameter must filter the folders accordingly: 'main' loads the 20 main varieties, 'new' loads the 10 new varieties, and 'all' loads all 30 varieties.
  6. Image Frame Selection: Within each selected folder, include *only* image files matching the specific regular expression or glob pattern `r0*.jpg`. Exclude all other frames.
  7. The `__getitem__` method must load the target image using PIL, apply an externally passed PyTorch transform, and return a tuple containing the transformed image tensor and its corresponding integer item label.
- 

#### Prompt 5: Task 1: Dataset Visualization Code

Prompt: Please write a compact, clean code snippet to verify and visualize our AppleDataset for Task 1. The code must:

1. Instantiate the AppleDataset using the 'Training' split and the 'main' class subset.
  2. Print out the exact count of unique main apple varieties found.
  3. Select exactly four sample apple item images from the dataset and render them in a horizontal, 1-by-4 matplotlib subplot layout.
  4. Title each subplot with its respective source item-folder name.
  5. Ensure the visual boundaries, margins, and image sizing are highly compressed and clean so that they fit nicely within standard boundaries during a direct notebook PDF export without truncating.
- 

#### Prompt 6: Task 1: Splitting and Data Pipeline Setup

Prompt: Let's set up the data loading pipeline for Task 1. Write the code to instantiate the following splits using our AppleDataset:

- Training dataset (split='Training', class\_subset='main')
- Test dataset (split='Validation', class\_subset='main')
- New-Support dataset (split='Training', class\_subset='new')
- New-Test dataset (split='Validation', class\_subset='new')

All splits must apply a composite transformation consisting of `transforms.Resize((64, 64))` and `transforms.ToTensor()`. Print out the total image count and the exact number of distinct classes present within each of these four splits. If helpful for standard training setups, map out PyTorch DataLoader instances. Performance Optimization: Because this dataset is relatively compact, implement a tensor caching mechanism to pre-load and store these images/labels directly in RAM memory to optimize subsequent training and validation iterations.

---

#### Prompt 7: Model: Compact CNN Embedding Model from Scratch

Prompt: Implement a custom, compact Convolutional Neural Network (CNN) embedding architecture from scratch using PyTorch. STRICT CONSTRAINT: Do not use any pre-trained networks, weights, or off-the-shelf architectures from torchvision.models or other libraries. The model must be constructed using basic building blocks:

- Sequential combinations of standard `nn.Conv2d`, `nn.BatchNorm2d`, `nn.ReLU`, and `nn.MaxPool2d` layers.
  - An `nn.AdaptiveAvgPool2d((1, 1))` layer to compress spatial dimensions.
  - A final `nn.Linear` projection layer that outputs a vector of size `EMBED_DIM = 128`.
- Inside the forward pass, apply strict L2-normalization on the final output embedding vector using `torch.nn.functional.normalize(x, p=2, dim=1)` before returning it.

---

### Prompt 8: Loss: Batch-Hard Triplet Loss Implementation

Prompt: Write a clean, optimized PyTorch implementation of the Batch-Hard Triplet Loss function. The function signature should be `batch_hard_triplet_loss(embeddings, labels, margin=0.3)`. The loss must satisfy the following implementation steps:

1. Compute the complete all-pairs Euclidean distance matrix across the batch using `torch.cdist`.
2. For every individual anchor sample in the batch, programmatically identify its hardest positive match (the maximum distance to another sample sharing the exact same label) and its hardest negative match (the minimum distance to any sample with a different label).
3. Ensure that the logic correctly handles or ignores anchors that do not possess valid positive counterparts in the current batch.
4. Use the traditional formulation:
$$\mathcal{L} = \max(0, D_{\{\text{anchor, positive}\}} - D_{\{\text{anchor, negative}\}} + \text{margin})$$
5. If a mini-batch happens to contain zero valid triplet combinations, return a differentiable zero tensor (`torch.tensor(0.0, requires_grad=True)`), or scaled from a model parameter) to prevent runtime exceptions or broken gradient chains.

---

### Prompt 9: Reusable Training & Evaluation Helpers + Task 1 Execution

Prompt: Provide a set of robust, highly reusable training and evaluation helper functions to organize our machine learning workflow, along with the execution logic for Task 1. Please implement:

1. `cache_labelled_dataset(dataset)`: Iterates through a dataset instance, pre-extracts all images and labels, and stacks them into unified, high-performance tensors.
2. `train_item_model(model, loader, optimizer, epochs, ...)`: A generic optimization loop that trains a model using our batch-hard triplet loss framework. Design this loop flexibly so it can handle pure item-level loss as well as optional joint item+family level objectives later.
3. `embed_tensor_images(model, images, batch_size)`: Extracts embeddings in safe mini-batch intervals to prevent GPU out-of-memory errors.
4. `nearest_neighbor_accuracy(support_embeddings, support_labels, query_embeddings, query_labels)`: Conducts evaluation by computing a 1-Nearest-Neighbor classification accuracy for queries against the support matrix.

Once implemented, instantiate a fresh CNN embedding model, train it on the Task 1 training dataset for exactly 15 epochs, and output the final seen-item accuracy (on the main test split) and unseen-item accuracy (on the new-test split).

---

### Prompt 10: Task 2: GardenDataset Implementation

Prompt: Implement a comprehensive, flexible PyTorch Dataset class called `GardenDataset` for Task 2. It must incorporate the following engineering specifications:

1. Accept the following structural arguments configuration:
  - `class_level`: One of ['item', 'family', 'both']
    - `subset`: Either ['train', 'test']
  - `family_subset`: One of ['main', 'new', 'all']
  - `item_subset`: One of ['main', 'new', 'all']
2. Family Derivation: Derive the family names programmatically from the raw dataset sub-folder text names by stripping away any final trailing numeric indicators or suffixes (e.g., "Apple 1"  $\rightarrow$  "Apple").
3. Mappings: Build a deterministic global index mapping system (`item_to_idx` and `family_to_idx`) across all folders found in the training directory.
4. Classification Categories:
  - New Families are explicitly defined as families that encompass exactly one solitary item folder across the source set.

- New Items are explicitly defined as the very first item folder discovered belonging to any family that contains a total of three or more item folders.
5. Subset Filtering: Apply the `family_subset` and `item_subset` arguments to filter the directories exactly as requested by isolating or excluding these designated main/new splits.
  6. Return Behavior: The `__getitem__` method must load the target image tensor and return labels depending on `class_level`. If `class_level="both"`, it must return a dictionary or tuple containing both the integer family index and the integer item index along with the image.

---

#### Prompt 11: Task 2: Sanity Checks and Dataset Visualization

Prompt: Please generate a concise testing and verification cell for the newly implemented GardenDataset. The block must perform the following actions sequentially:

1. Run explicit hard assertions verifying that the main training split contains exactly 15 unique families and 69 unique item categories.
2. Assert that the designated new-item test split and new-family test split are non-empty.
3. Verify that the file paths pointing to the support training samples are completely disjoint from the evaluation test file paths.
4. Print out a summary detailing the respective sizes and sample counts of each configuration scenario.
5. Plot a clean, 1-by-4 grid visualization of four sample images drawn from the GardenDataset with compact formatting suited for publication/PDF printing.

---

#### Prompt 12: Task 2: Cached kNN Evaluation Logic for 4 Scenarios

Prompt: Write a highly optimized, modular evaluation block that calculates metrics across all four required Task 2 scenarios using cached embeddings. The evaluation routine must systematically address these scenarios:

- Scenario 1 (Seen-Item Recognition): Support set = Main Train images; Query set = Main Test images. Evaluated at the *Item* level.
- Scenario 2 (Seen-Family Recognition): Support set = Main Train images; Query set = Main Test images. Evaluated at the *Family* level.
- Scenario 3 (Unseen-Item Novelty): Support set = Main Train images + New Items; Query set = New-Item Test images. Evaluated at the *Item* level.
- Scenario 4 (Unseen-Family Novelty): Support set = Main Train images + New Families; Query set = New-Family Test images. Evaluated at the *Family* level.

The evaluation function must extract embeddings for all relevant sets, execute a  $k$ -Nearest Neighbors ( $k\text{-NN}$ ) classification search evaluated for  $k \in \{1, 3, 5\}$ , and return the precise classification accuracy and target test set sample size for each run. Provide an option to cache or return raw classification predictions to generate downstream visual analytics.

---

#### Prompt 13: Task 2: Training and Diagnostic Visualization Outputs

Prompt: Please write the training and diagnostic execution loop for Task 2. Your script must:

1. Initialize a fresh, un-pretrained instance of our CNN embedding model architecture.
2. Train this model from scratch exclusively on the main GardenDataset training split using our `batch_hard_triplet_loss` optimized on item-level labels for exactly 15 epochs.
3. Immediately evaluate the trained model across the four evaluation scenarios using  $k \in \{1, 3, 5\}$  and display the output metrics in a clean, compact Markdown results table.
  4. Generate a 3-part diagnostic visual layout:
    - A line plot illustrating the training loss over the 15 epochs.
  - A clear horizontal or vertical bar chart plotting the Scenario 1 *item accuracy breakdown calculated per distinct fruit family*.
  - A comprehensive family-level confusion matrix visualization for Scenario 2.
5. Keep all plots compact and clean to guarantee a high-quality, professional PDF compilation output.

---

#### Prompt 14: Task 3: Combined Loss Implementation and Comparative Evaluation

Prompt: Let's implement Task 3 to evaluate multi-task learning using combined structural hierarchies. STRICT EXPERIMENTAL CONSTRAINTS: Do not write any hyperparameter search routines, optimization loops, or alpha sweeps that access or test against the validation sets. Use the fixed hyperparameters specified by the requirements:

TASK3\_ALPHA = 0.25

TASK3\_FINAL\_K = 1

TASK3\_EPOCHS = 15

Please write the code to:

1. Train exactly one single joint embedding model entirely from scratch.
2. Configure the joint training optimization objective to use a composite loss function defined as:

$$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{item}} + \alpha \cdot \mathcal{L}_{\text{family}}$$

Both  $\mathcal{L}_{\text{item}}$  and  $\mathcal{L}_{\text{family}}$  must utilize our batch-hard triplet loss formulation, making sure the baseline item-level objective remains intact. 3. Train the model for exactly 15 epochs on the main training split. 4. Evaluate the model at  $k=1$  across the identical four scenarios defined in Task 2. 5. Print a clear section header string: "Final Task 3 fixed-alpha results". 6. Construct and display a direct comparative results table highlighting the percentage-point performance deltas ( $\Delta$ ) achieved by Task 3 relative to the baseline performance from Task 2.

---

#### Prompt 15: Task 4: Black-Pixel Corruption Transform

Prompt: Prepare the data corruption framework for Task 4. Write a custom PyTorch tensor transform or utility function that implements a spatial black-pixel corruption mask. The transform must conform to these specifications:

1. It must execute as a downstream step *after* the images have already been resized to  $64 \times 64$  and converted into floating-point PyTorch tensors.
2. It must accept a parameter defining the target corruption fraction ( $f$ ).
3. It must randomly select a proportion of spatial pixel coordinates  $(x, y)$  equivalent to fraction  $f$  and forcefully overwrite their values to  $0.0$  (black) across all three RGB channels simultaneously.
4. If the passed fraction  $f = 0.0$ , it must return an uncorrupted, independent clone of the original tensor.
5. For very low positive fractions (e.g.,  $f = 0.01$ ), include an internal safety guard or floor limit ensuring that at least one single pixel is actively corrupted.

---

#### Prompt 16: Task 4: Prototype Embedding Anomaly Detector

Prompt: Implement the baseline embedding-based anomaly detection pipeline for Task 4. The implementation steps are:

1. Use the joint embedding model trained during Task 3 in a frozen inference state.
2. Feed all clean, normal training samples from the GardenDataset main training split through the model to extract their embeddings.
3. Class Prototypes: Compute the spatial centroid (average vector) for each item class based on its clean training embeddings. Apply strict L2-normalization on these resulting class prototype vectors.
4. Anomaly Scoring: Define the anomaly score of any incoming query image tensor as:
$$\text{Score} = 1 - \max(\text{Cosine Similarity to any Item Prototype})$$
5. Calibration: Compute the anomaly scores across all uncorrupted, clean normal test images. Find the exact 90th percentile threshold of these clean scores. This sets our operating point to yield approximately a 10% clean false-positive rate (FPR).
6. Evaluation: Apply this calibrated threshold to evaluate test sets corrupted at 1%, 5%, and 10% black-pixel fractions, and calculate their respective anomaly detection rates.

---

#### Prompt 17: Task 4: Clean-to-Clean Convolutional Autoencoder Anomaly Detector

Prompt: Implement the second anomaly detector for Task 4: a custom Convolutional Autoencoder built from scratch using PyTorch. CRITICAL ARCHITECTURAL & TRAINING CONSTRAINTS: > \* Do not train this model as a denoising autoencoder.

- Do not present, inject, or expose any corrupted or noisy images to the autoencoder at any stage during its training loop. It must learn exclusively on a clean-to-clean reconstruction mapping.

Requirements:

1. Architecture: Create an Encoder block containing standard `nn.Conv2d`, `nn.ReLU`, and `nn.MaxPool2d` layers. Create a matching Decoder block utilizing spatial upsampling (e.g., `nn.Upsample`), `nn.Conv2d`, `nn.ReLU`, and a final bounding `nn.Sigmoid` activation.
2. Optimization: Train the autoencoder from scratch on the completely clean main GardenDataset training split using an L1 reconstruction loss (`nn.L1Loss`) for exactly 25 epochs.
3. Anomaly Score Formulation: For any query image, compute its absolute reconstruction pixel residuals ( $\|x - \hat{x}\|$ ). Average these residuals across the 3 RGB channels to produce a single spatial error map. Flatten this map into a 1D spatial array, isolate the top 1% largest pixel residual values, and compute their mean to establish the image's final anomaly score.

---

#### Prompt 18: Task 4: Shared Corrupted Test Tensors Pipeline

Prompt: To guarantee a perfectly fair, rigorous benchmark comparison between our prototype embedding anomaly detector and our convolutional autoencoder detector, they must evaluate identical corrupted data.

Please write a data setup block that performs the following steps:

1. Set our global deterministic seed environment (`SEED = 6`).
2. Isolate the target Task 4 normal test image collection.
3. Precompute a centralized dictionary of corrupted tensors up-front:  
`corrupted_by_fraction = {f: corrupt_batch(task4_test_images, f) for f in [0.01, 0.05, 0.10]}`
4. Pass this identical dictionary reference directly into both independent detector testing pipelines so that there is zero stochastic variance in the evaluation inputs.

---

#### Prompt 19: Task 4: Compilation and Result Presentation

Prompt: Let's finalize the reporting for Task 4. Please write code to compile the final metrics and generate the necessary plots. Requirements:

1. Construct a clean, scannable Markdown summary table displaying the performance of our detectors side by side. The table must feature the following exact column layout: [Method, Threshold, Clean False-Positive Rate, 1% Corruption Detection Rate, 5% Corruption Detection Rate, 10% Corruption Detection Rate]
2. Provide exactly two rows for this table, labeled precisely as "Prototype embedding" and "Autoencoder".
3. Generate a line plot showing the training loss history of the Convolutional Autoencoder over its 25-epoch cycle.
4. STRICT COMPLIANCE CHECK: Double-check all text blocks, code annotations, labels, and markdown headers surrounding this task. Ensure that the specific phrase "Denoising autoencoder" does not appear anywhere within the code or text.

---

#### Prompt 20: Code Review / Robustness of notebook\_builder.py

Prompt: Please perform a comprehensive code review and quality verification check on our `notebook_builder.py` script before we execute the pipeline. Verify that the generation code is completely bulletproof against the following rules:

1. It must successfully parse `ass1_empty_workbook.ipynb` and output `ass1_final_submission.ipynb`.
2. The output notebook must maintain exactly the same total number of cells, identical cell indexing order, and identical cell type designations (markdown vs. code) as the template.
3. It must preserve our filled group identification metadata and member heading block without modifications.
4. It must strip out all execution history, runtime counts, and tracebacks, ensuring a clean slate.



5. Academic Integrity Check: Confirm that no code cells inject answers to the conceptual text questions. There must be zero textual explanation answers embedded within the generated code cells beyond standard structural headers.
  6. Architecture Constraint: Confirm that absolutely no code block pulls pre-trained weights or external parameters.
- 

#### Prompt 21: Static Preflight Checker Script

Prompt: I want to catch any potential compliance or formatting errors before runtime. Please write a standalone python verification script named `preflight_checker.py` that opens our generated `ass1_final_submission.ipynb` file and statically checks the following conditions:

1. Validates that the notebook file parses as correct, uncorrupted JSON structure.
  2. Iterates through all populated code blocks and runs Python's built-in `compile()` function on the cell contents to check for syntax errors.
  3. Confirms that the cell counts and types perfectly mirror the original skeleton notebook.
  4. Scans strings to verify that the dataset handle contains the correct version string (`versions/87`).
  5. Flags an error if any prohibited terms or code constructs related to pre-trained weights are present (e.g., `pretrained=True`, `weights=`, etc.).
  6. Verifies that Task 3 does not contain any alpha sweeps or validation-set hyperparameter selection logic.
  7. Confirms that Task 4 trains the autoencoder purely on clean normal data and does not follow a denoising setup.
  8. Statically verifies that both Task 4 detectors process the identical `corrupted_by_fraction` dictionary.
- 

#### Prompt 22: Clean Kernel Execution using nbclient

Prompt: Now that our notebook generation and static preflight checks pass, let's execute the entire submission from scratch. Write an automated execution script utilizing the `nbclient` and `nbconvert` APIs. The script must execute the following workflow:

1. Open `ass1_final_submission.ipynb` and execute it from a completely clean kernel context.
2. Set the client parameter to stop execution immediately if any unhandled error or exception is encountered.
3. Save the successfully executed notebook overwriting its state with all computed cells, logs, and embedded matplotlib outputs preserved.
4. Print out a runtime execution summary report directly to stdout detailing:
  - Total execution duration.
  - Total number of successfully run code cells.
  - Detailed error tracebacks (if any failure occurred).
- Automated extraction and summary printing of the final key metrics achieved for Task 1, Task 2, Task 3, and Task 4.

Note: Do not dynamically edit or modify the code contents during this run unless a direct runtime exception occurs.

---

#### Prompt 23: PDF Export Workflow

Prompt: Our executed notebook contains all the final outputs. I need to export this completed notebook file into a presentation-ready PDF document for submission. Please write an automated export routine (using `nbconvert` to HTML combined with Playwright, or another dependable headless printing engine) that generates our submission PDF. Ensure the export adheres to these output criteria:

1. The output file size must be optimized to stay safely under the 25 MB limit.
2. All charts and text tables must remain completely visible, properly sized, and un-truncated within standard page dimensions.
3. Double-check that the following mandatory elements are fully rendered and clearly legible:
  - Task 2: Per-family item accuracy bar chart.
  - Task 2: Family-level confusion matrix.

- Task 3: Fixed-alpha comparative delta performance table.
  - Task 4: Parallel anomaly detection comparison table.
- 

Prompt 24: Senior Code Reviewer / Assignment Grader Persona Review

Prompt: Act as a strict senior code reviewer and academic assignment grader. I want you to perform a final audit on our executed notebook (ass1\_final\_submission.ipynb) and the exported PDF file against the official assignment rubric. Review the implementation and verify that:

1. All Tasks 1 through 4 are fully implemented without missing elements.
2. Every model architecture is explicitly built from scratch without utilizing hidden pre-trained weights.
3. Task 3 respects the fixed parameters ( $\alpha = 0.25$ ,  $k = 1$ , 15 epochs) and preserves the original item-level optimization pathway without test-set tuning.
4. Task 4's convolutional autoencoder is trained strictly under clean-to-clean constraints without exposure to noise.
5. Both Task 4 anomaly detectors share the exact same precomputed corrupted data structures.
6. All required plots, matrix graphics, and comparative tables are properly rendered in the final PDF file.

Please skip general praise or style tips and return only critical must-fix issues that would risk losing points under strict grading criteria. If the submission is fully compliant, respond with a clear confirmation statement.